

ÉCOLE CENTRALE DE NANTES

SMP — TP10 | Projet noté

CAHIER DES CHARGES

Les Aventuriers du Rail – Mon Premier Voyage

Implémentation C++ d'un jeu de société multijoueur

Champ	Détail
Projet	Les Aventuriers du Rail – Mon Premier Voyage
Équipe	Vincent, David, Angélique, Riana
Langage	C++
Interface	Texte (console)
Joueurs	2 à 4
Dépôt	GitHub — branche main

1. Objectif du Projet

L'objectif est d'implémenter en C++ une version simplifiée du jeu Les Aventuriers du Rail – Mon Premier Voyage. Le programme doit permettre à 2 à 4 joueurs de s'affronter tour à tour via une interface textuelle. L'état de la partie doit être lisible à tout moment depuis la console.

2. Contexte et Règles du Jeu

2.1 Description générale

Le plateau représente un réseau de villes reliées par des voies ferrées colorées. Chaque joueur tente de relier les paires de villes indiquées sur ses tickets en posant ses wagons sur les voies disponibles. Le but est d'être le premier à valider 6 tickets.

2.2 Ressources de chaque joueur

- 20 wagons d'une couleur parmi : rouge, jaune, vert, bleu
- 4 cartes Train distribuées en début de partie (6 couleurs + locomotives jokers)
- 2 tickets distribués en début de partie (chaque ticket relie deux villes)

2.3 Mise en place

- Chaque joueur choisit une couleur et reçoit 20 wagons
- Les 72 cartes Train sont mélangées ; 4 sont distribuées à chaque joueur, le reste forme la pioche
- Les 32 tickets sont mélangés ; 2 sont distribués à chaque joueur, le reste forme la pioche
- Les cartes des joueurs restent secrètes des autres joueurs

2.4 Déroulement d'un tour

A chaque tour, un joueur choisit exactement une des trois actions suivantes :

- Piocher deux cartes Train depuis la pioche
- Poser ses wagons sur une voie ferrée (la capturer)
- Passer son tour : défausser ses deux tickets actuels et en piocher deux nouveaux

2.5 Règles de prise de voie ferrée

- Le joueur doit jouer autant de cartes Train de la couleur de la voie que sa longueur
- Les locomotives sont des jokers et remplacent n'importe quelle couleur
- Les cartes jouées sont immédiatement défaussées
- Le joueur place ses wagons sur la voie : une voie ne peut être prise qu'une seule fois
- Pour les voies doubles (parallèles), un même joueur ne peut pas prendre les deux

2.6 Validation d'un ticket

Un ticket est réussi lorsque les deux villes qu'il indique sont reliées par un chemin continu de wagons appartenant uniquement au joueur. Le joueur annonce Ticket !, place le ticket face visible, puis pioche immédiatement un nouveau ticket.

2.7 Grande Traversée

Le premier joueur à relier une ville de la cote ouest à une ville de la cote est remporte la carte Bonus Grande traversée, qui compte comme un ticket réussi supplémentaire.

2.8 Fin de partie

- La partie s'arrête lorsqu'un joueur réussit son 6e ticket
- Ou lorsqu'un joueur pose son dernier wagon
- Le gagnant est le joueur ayant validé le plus grand nombre de tickets

2.9 Contenu du jeu

- 72 cartes Train : 10 par couleur (jaune, vert, blanc, noir, bleu, rouge) + 12 locomotives
- 32 tickets
- 4 cartes Bonus Grande traversée
- 1 ticket d'or Golden Ticket
- 80 wagons plastique (20 par couleur/joueur)

3. Exigences Fonctionnelles

3.1 Initialisation de la partie

- Demander le nombre de joueurs (entre 2 et 4)
- Demander à chaque joueur de choisir une couleur parmi les 4 disponibles (enumCouleur)
- Charger le plateau depuis le fichier CSV (villes et voies ferrées)
- Charger les tickets depuis le fichier CSV dédié
- Mélanger et distribuer 4 cartes Train par joueur
- Mélanger et distribuer 2 tickets par joueur

3.2 Gestion des pioches

- Pioche de cartes Train : 2 cartes par action de pioche
- Pioche de tickets : 2 tickets lors d'un remplacement (action passer son tour)
- Si la pioche est vide, mélanger la défausse et la remettre en jeu
- La méthode estPiocheVide() vérifie l'état de chaque pioche

3.3 Pose de wagons

- Vérifier la disponibilité de la voie (estDispo())
- Vérifier que le joueur ne possède pas déjà la voie parallèle
- Vérifier que le joueur dispose du nombre suffisant de cartes de la bonne couleur
- Déduire les wagons et cartes utilisés du joueur
- Marquer la voie comme occupée par la couleur du joueur

3.4 Vérification automatique des tickets

- Après chaque pose de wagons, vérifier automatiquement tous les tickets en cours
- Utiliser un algorithme BFS ou DFS sur le graphe des voies possédées par le joueur
- Annoncer Ticket ! et déclencher la pioche d'un nouveau ticket si validé
- Vérifier la condition Grande Traversée (côte est/ouest connectées)

3.5 Affichage de l'état de la partie

- Afficher le plateau : matrice d'adjacence avec couleurs des voies occupées
- Afficher la main complète de chaque joueur (cartes Train, tickets, wagons restants)
- Afficher les tickets validés face visible
- L'état global doit être consultable à tout moment

3.6 Détection de fin de partie

- Détecter automatiquement la validation du 6e ticket d'un joueur
- Détecter automatiquement la pose du dernier wagon d'un joueur

- Annoncer clairement le gagnant en fin de partie

4. Exigences Non Fonctionnelles

- Interface entièrement textuelle (console), sans interface graphique
- Code documenté (commentaires de type Doxygen ou similaire)
- Code testé avec des jeux d'essais illustrant chaque fonctionnalité clé
- Versionné sous GitHub, branche main du groupe
- README.md avec instructions claires de compilation et d'exécution
- Chargement des données (plateau, tickets) obligatoirement depuis des fichiers CSV
- Séparation claire des responsabilités entre les classes (principe SRP)

5. Architecture Orientée Objet

5.1 Vue d'ensemble des classes

Classe	Responsable	Rôle principal
Jeu/main	<i>Vincent</i>	Logique globale de la partie (tours, fin, gagnant)
Plateau	<i>David</i>	Graphe du réseau de villes et voies ferrées
Pioche	<i>David</i>	Gestion des decks de cartes Train et tickets
Carte	<i>David</i>	Classe de base pour les cartes
Ticket	<i>David</i>	Carte objectif : vérifie si le trajet est réalisé
CTrain	<i>David</i>	Carte Train colorée ou locomotive (joker)
Joueur	<i>Angélique/ Vincent</i>	Ressources et actions d'un joueur
VoieFerre	<i>Riana</i>	Arête du graphe (voie entre deux villes)
NoeudVille / Ville	<i>Riana</i>	Sommet du graphe (une ville)

5.2 Spécification détaillée des méthodes

Classe : Jeu — Responsable : *Vincent*

Méthode / Signature	Description
<code>Jeu(int nbJoueur)</code>	Demande la couleur souhaitée à chaque joueur (parmi enumCouleur 1-4), crée les instances associées (Plateau, Joueurs, Pioches)
<code>void partie()</code>	Boucle principale : gère l'enchaînement des tours, les actions des joueurs et les conditions de fin
<code>bool estPartieFinie()</code>	Teste si la partie est terminée (6e ticket valide ou dernier wagon pose) et annonce le gagnant
<code>Getters</code>	Accesseurs sur les attributs (joueurs, plateau, pioches, etc.)

Classe : Plateau — Responsable : *David*

Méthode / Signature	Description
<code>Plateau()</code>	Charge le graphe (villes + voies) depuis le fichier CSV fourni
<code>void affichePlateau()</code>	Affiche la matrice d'adjacence avec listes pour voies multiples et couleurs des voies occupées
Getters	Accesseurs sur les attributs du graphe (villes, voies, adjacence)

Classe : Pioche — Responsable : *David*

Méthode / Signature	Description
<code>Pioche(string type)</code>	Crée les instances de cartes selon le type (Ticket ou CTrain) et applique les règles associées a chaque type
<code>void melanger()</code>	Mélange les cartes aléatoirement (algorithme de Fisher-Yates recommandé)
<code>bool estPiocheVide()</code>	Retourne vrai si la pioche ne contient plus aucune carte
Getters	Accesseurs sur le contenu de la pioche

Classe : Carte — Responsable : *David*

Méthode / Signature	Description
<code>Carte()</code>	Constructeur de base (classe parente, potentiellement abstraite)

Classe : Ticket — Responsable : *David*

Méthode / Signature	Description
<code>bool estRealise()</code>	Vérifie qu'un chemin continu de wagons du joueur relie les deux villes du ticket (BFS/DFS sur le sous-graphe du joueur)

Classe : CTrain — Responsable : *David*

Méthode / Signature	Description
<code>Couleur_e getCouleur()</code>	Retourne la couleur de la carte Train (ou l'indicateur Locomotive pour les jokers)

Classe : Joueur — Responsable : *Angélique/Vincent/Riana*

Méthode / Signature	Description
<code>Joueur(int couleur)</code>	Constructeur : initialise toutes les ressources a zero (mainCarte, mainTicket, mainWagon vides)
<code>void piocher(Pioche type)</code>	Pioche depuis la pioche indiquée et ajoute les cartes a la main du joueur selon le type (CTrain ou Ticket)
<code>void poserWagon(enumVille a,</code>	Pose des wagons sur la voie (a,b,c) : vérifie disponibilité, voie parallèle, et ressources suffisantes

<code>enumVille b, enumCouleur c)</code>	
<code>void afficherMain()</code>	Affiche la main complète : cartes Train (mainCarte), tickets en cours (mainTicket), wagons restants
<code>void defausserCarte(Carte c)</code>	Retire une carte de la main du joueur et la place dans la défausse appropriée
Getters	Accesseurs sur les ressources du joueur (main, wagons, tickets validés)

Classe : VoieFerre — Responsable : *Riana*

Méthode / Signature	Description
<code>VoieFerre(enumVille v1, enumVille v2)</code>	Constructeur : crée une arête entre v1 et v2. Clé primaire composite (v1, v2, couleur) pour gérer les voies parallèles
<code>bool estDispo()</code>	Vérifie que la voie est libre (pas encore prise) en tenant compte des voies parallèles

Classe : NoeudVille / Ville — Responsable : *Riana*

Méthode / Signature	Description
<code>void afficheListeVille()</code>	Affiche la liste complète de toutes les villes du plateau (enumVille)
<code>void afficheAdjacent()</code>	Affiche les villes adjacentes à la ville sélectionnée dans le graphe
<code>bool estAdjacent()</code>	Teste si deux villes sont directement adjacentes dans le graphe du plateau

6. Données et Fichiers CSV

6.1 Fichier des routes (plateau)

Ce fichier décrit l'ensemble des voies ferrées. Chaque ligne représente une voie avec ses propriétés :

- Ville de départ et ville d'arrivée
- Couleur de la voie (enumCouleur)
- Longueur de la voie (nombre de wagons requis)
- Indicateur de voie double/parallèle

6.2 Fichier des tickets

Ce fichier liste les 32 tickets du jeu. Chaque ligne représente un ticket :

- Ville de départ et ville d'arrivée
- Indicateur Grande Traversée (si applicable — côte est / côte ouest)

6.3 Remarque sur les enums

La création automatique d'un type enum en C++ n'étant pas automatisable depuis un CSV, les villes sont gérées via un `std::vector` chargé dynamiquement depuis le fichier, ce qui offre plus de flexibilité en cas de modification du plateau.

7. Découpage des Tâches et Planning

Phase 1 — Initialisation (tous)

- Rédaction du cahier des charges
- Définition des règles, contraintes et choix techniques

Phase 2 — Conception (tous)

- Réalisation du diagramme UML de classes
- Réalisation du diagramme de cas d'utilisation
- Rédaction de l'algorithme principal de la boucle de jeu

Phase 3 — Programmation (répartie)

- Programmation des classes dans des branches Git séparées
- Tests unitaires de chaque classe
- Intégration dans la branche main
- Programmation de la boucle de jeu (`void partie()`)
- Tests d'intégration

Phase 4 — Finalisation (tous)

- Réévaluation et mise à jour du diagramme UML
- Tests complets de la partie
- Rédaction de la documentation et du README
- Constitution des jeux d'essais illustratifs

8. Problèmes Rencontrés et Décisions Techniques

Clé VoieFerre

La clé primaire est composée de 3 éléments (`v1`, `v2`, couleur) et non 2, pour gérer correctement les voies parallèles de même couleur.

Enum -> Vector

Les villes ne sont pas stockées dans un enum statique : elles sont chargées depuis le CSV dans un `std::vector` pour plus de flexibilité.

Tickets gagnés

Chaque joueur possède un vecteur de tickets validés (face visible) distinct de

	sa main courante de tickets en cours.
Renommage Ville	La classe NoeudVille a été renommée Ville et l'attribut ville renommé ville_ pour respecter les conventions C++ (attributs privés avec underscore).
Signature piocher()	La méthode piocher() du Joueur ne prend plus un paramètre nbAPiocher : le nombre de cartes est implicite selon le type de pioche (1 ou 2).
Défausse explicite	La méthode defausserCarte(Carte c) a été ajoutée a la classe Joueur pour gérer explicitement la défausse de cartes Train ou de tickets.